Data Structures – Basic Terminology

1. **Data**: Data refers to raw facts and figures that can be processed and analyzed to draw conclusions and make decisions. It can be in any form, such as numbers, text, images, or audio. Data is usually organized in a specific structure to make it more meaningful and easier to process.
2. **Entity**: An entity is an object or concept that can be distinctively identified and has a specific role in a system or process. It can refer to a real-world object, such as a person, a place, or a thing, or a virtual object, such as a record in a database. Entities are typically used to model the objects or concepts that are relevant to a particular problem or domain.
3. **Information**: Information is the result of processing data to extract meaning and knowledge. It is data that has been organized and presented in a meaningful way, and it provides insights and context that can be used to make decisions and solve problems. Information is typically more valuable than raw data, as it provides a higher level of understanding and can be used to drive action.

In other words, data is a collection of facts, figures, and statistics, while information is data that has been transformed into a more useful form. An entity is a real-world object or concept that can be represented as data, and information can be derived from that data by processing and organizing it.

Abstract Data Type (ADT) is a way of defining the structure and behavior of a specific type of data. It provides a blueprint or a template for creating new data types, and it is defined independently of any particular implementation.

In other words, ADT is like a blueprint for a house. Just as a blueprint provides a plan for the structure and layout of a house, ADT provides a plan for the structure and behavior of a specific type of data. The blueprint of the house can be used to build the house in different ways, with different materials, and in different locations. Similarly, ADT can be used to implement the data type in different programming languages and using different data structures. An ADT defines what operations can be performed on the data and what these operations do, but it does not specify how the data is actually stored or managed. This allows for multiple different implementations of the same ADT, each with its own unique features and trade-offs.

For example, the ADT for a stack could be defined as follows:

- A stack is a collection of items
- The items in a stack are stored in a specific order, called Last-In-First-Out (LIFO)
- The operations that can be performed on a stack include:
    - Push: Add an item to the top of the stack
    - Pop: Remove the item from the top of the stack
    - Peek: Look at the item at the top of the stack without removing it

These definitions specify the structure and behavior of a stack, but they do not specify how the stack should be implemented. The implementation of the stack, such as using an array or a linked list, is left to the programmer.

In summary, ADT is a way of defining a specific type of data, specifying its structure and behavior, and providing a blueprint for creating new data types. It is a powerful tool for designing and implementing complex data structures and algorithms.

Algorithm design techniques are methods for developing efficient, effective, and scalable algorithms to solve computational problems. There are several commonly used algorithm design techniques, including:

1. Brute Force: This technique involves generating all possible solutions to a problem and checking each one until a correct solution is found. This approach can be slow and inefficient, but it can be useful for solving smaller problems or for testing other algorithms.
2. Divide and Conquer: This technique involves breaking down a problem into smaller sub-problems and solving each sub-problem individually. The solutions to the sub-problems are then combined to solve the overall problem. This approach can be very efficient and can lead to algorithms with fast running times.
3. Dynamic Programming: This technique involves breaking down a problem into smaller sub-problems and solving each sub-problem only once, storing the solutions to each sub-problem in a table for later use. This approach can be very efficient and can lead to algorithms with fast running times, but it requires a significant amount of memory.
4. Greedy Algorithms: This technique involves making a series of locally optimal choices in an attempt to find a globally optimal solution. Greedy algorithms can be very fast, but they may not always find the best solution to a problem.
5. Backtracking: This technique involves exploring a search space by incrementally building a solution and undoing the last step if it leads to an incorrect solution. This approach can be very effective for solving problems with multiple solutions, but it can be slow and inefficient.
6. Randomized Algorithms: This technique involves using randomness as a way to solve a problem or find a solution. Randomized algorithms can be very fast and effective, but they may not always find the correct solution.

These are just a few of the many algorithm design techniques that are used in computer science. The best technique for a particular problem depends on the specific requirements of that problem, including its size, complexity, and available resources.

A **data structure** is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. A data structure determines how data is stored, accessed, and manipulated within a program.

There are several types of data structures, including:

1. Arrays: An array is a collection of elements of the same data type, stored in contiguous memory locations. Arrays are used to store data that can be accessed randomly by its index.
2. Linked Lists: A linked list is a collection of elements, called nodes, where each node contains a reference to the next node in the list. Linked lists are used to store data that can be accessed sequentially, with the ability to insert and delete elements at any point.

3. Stacks: A stack is a linear data structure that operates on the Last In First Out (LIFO) principle. It is used to store data that can be accessed only at the top of the stack.
4. Queues: A queue is a linear data structure that operates on the First In First Out (FIFO) principle. It is used to store data that can be accessed only at the front of the queue.
5. Trees: A tree is a hierarchical data structure that is composed of nodes, where each node has a parent node and one or more child nodes. Trees are used to store hierarchical data, such as family trees, file systems, or decision trees.
6. Graphs: A graph is a collection of nodes and edges, where the edges represent connections between the nodes. Graphs are used to store data that has relationships between elements, such as social networks, road networks, or computer networks.
7. Hash Tables: A hash table is a data structure that uses a hash function to map keys to indices in an array. Hash tables are used to store data that can be accessed quickly by a key, such as dictionaries, phone books, or databases.

In conclusion, a data structure is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. There are several types of data structures, including arrays, linked lists, stacks, queues, trees, graphs, and hash tables. The choice of data structure depends on the specific requirements of the problem being solved, such as the size of the data, the relationships between elements, and the desired access patterns.

Basic operations on data structures are fundamental actions that can be performed on the elements within these structures. These operations allow for the manipulation, traversal, and management of the data contained in the structures. Here are the common operations performed on various data structures:

Common Operations

Traversal:

Accessing each element of the data structure to process it.

Example: Traversing an array from start to end, visiting all nodes of a linked list, or traversing a tree using in-order, pre-order, or post-order traversal.

Insertion:

Adding an element to the data structure.

Example: Adding an element to an array, a node to a linked list, or a value to a binary search tree.

Deletion:

Removing an element from the data structure.

Example: Deleting an element from an array, removing a node from a linked list, or deleting a node from a binary search tree.

Searching:

Finding an element within the data structure.

Example: Searching for a value in an array, linked list, or searching for a node in a binary search tree.

Sorting:

Arranging the elements in a specific order.

Example: Sorting elements of an array in ascending order using sorting algorithms like quicksort or mergesort.

Merging:

Combining two data structures into one.

Example: Merging two sorted arrays or linked lists into a single sorted array or linked list.

Here is a **comparison between linear and non-linear data structures** in a tabular format:

Feature	Linear Data Structures	Non-Linear Data Structures
Definition	Data elements are arranged in a sequential manner.	Data elements are arranged in a hierarchical manner.
Examples	Arrays, Linked Lists, Stacks, Queues	Trees, Graphs
Memory Utilization	May require contiguous memory allocation (e.g., arrays)	Can utilize non-contiguous memory allocation
Traversal	Single-level traversal (one element after another)	Multi-level traversal (parent-child relationships)
Complexity	Generally simpler to implement and understand	More complex due to hierarchical relationships
Insertion/Deletion	Easier in some structures (e.g., linked lists)	Can be complex, especially in balanced structures
Search Operation	Linear search (O(n)), Binary search in sorted structures (O(log n))	Varies; e.g., Binary Search Tree (O(log n)), Graph (varies)
Use Cases	Simple sequential data storage, stack and queue operations	Representing hierarchical data, complex relationships, networks
Access Time	Constant time (O(1)) for arrays; Linear time (O(n)) for linked lists	Varies; O(log n) for balanced trees, can be O(1) to O(n) for graphs
Examples of Operations	Array index access, stack push/pop, queue enqueue/dequeue	Tree traversal (in-order, pre-order), graph traversal (DFS, BFS)

In conclusion, linear data structures are straightforward and store elements in a sequence. On the other hand, non-linear data structures are complex and store elements in a non-sequential manner, forming complex relationships between elements. The choice of data structure depends on the specific requirements of the problem being solved, such as the size of the data, the relationships between elements, and the desired access patterns.

NOTE: Trees and graphs are considered non-linear data structures despite being implemented using arrays and linked lists because their logical organization and the relationships between their elements are inherently hierarchical and non-sequential.